MEDIVAULT

DOSSIER DE CONCEPTION

MediVault

Votre santé, en sécurité, dans un seul dossier.

Validation intermédiaire 1

Applications Web Distribuées (MT-41) · Architecture microservices



ÉQUIPE

Keystone

Classe 4DAS_I4

MEMBRES

Eya · Amine · Fedia · Aziz · Taha · Yazid

ANNÉE UNIVERSITAIRE

2026 – 2027

ESPRIT

Sommaire

> *MediVault donne à chaque patient un dossier médical numérique unique, ouvert aux professionnels uniquement avec son accord : les prescriptions sont vérifiées, chaque ordonnance n'est délivrée qu'une seule fois et les résultats d'analyses arrivent directement dans le dossier.*

Équipe Keystone — 4DAS_I4
ESPRIT · 2026-2027

01 **Présentation du projet** — Contexte, problématique, acteurs, fonctionnement et scénario — 3

02 **Diagramme de cas d'utilisation** — Acteurs et cas d'utilisation de la plateforme — 7

03 **Découpage en microservices** — Services, bases de données, technologies, user-service — 8

04 **Architecture technique globale** — Vue d'ensemble : Gateway, Eureka, Config Server, RabbitMQ — 10

05 **Diagramme de classes** — Classes par microservice, énumérations et règles de gestion — 11

06 **Interactions entre microservices** — Communication synchrone (OpenFeign) et asynchrone (RabbitMQ) — 13

07 **Contrats d'API et événements** — Endpoints REST, codes HTTP et événements métier — 15

08 **Organisation, planning et risques** — Répartition des tâches, planning et risques — 17

1 Présentation du projet

1.1 Contexte

Aujourd'hui, chaque professionnel de santé (généraliste, spécialiste, laboratoire, pharmacie) conserve ses propres informations sur le patient, le plus souvent sur papier. Le patient doit transporter lui-même ses ordonnances, ses analyses et ses comptes rendus, et il en perd ou en oublie régulièrement.

1.2 Problématique

- **Informations dispersées** : aucun professionnel n'a une vue complète de l'historique du patient.
- **Erreurs de prescription** : un médecin peut prescrire un médicament auquel le patient est allergique, ou incompatible avec un traitement en cours.
- **Ordonnances papier** : une même ordonnance peut être utilisée plusieurs fois ou falsifiée.
- **Absence de contrôle** : le patient ne décide pas qui peut consulter ses données médicales.

> **Comment centraliser le dossier médical du patient de façon sûre, tout en réduisant les erreurs de prescription ?**

1.3 Solution proposée

MEDIVAULT (*medi* + *vault* : le « coffre-fort » médical du patient) est une plateforme web en architecture microservices qui donne à chaque patient un **dossier médical numérique unique**, accessible aux professionnels **uniquement avec son accord**.

- **Vérification automatique** des allergies (par famille thérapeutique) et des interactions médicamenteuses lors de la prescription ;
- **Ordonnance numérique** délivrée une seule fois en pharmacie (totale ou partielle) ;
- **Résultats d'analyses** transmis directement par le laboratoire dans le dossier ;
- **Prise de rendez-vous** et **notifications** pour chaque événement important.

1.4 Acteurs

- **Patient** : consulte son dossier, gère les accès, prend rendez-vous, reçoit les notifications.
- **Médecin généraliste** : consulte le dossier (si autorisé), ajoute des consultations, rédige des ordonnances, gère son agenda.
- **Pharmacien** : recherche et délivre les ordonnances.
- **Laborantin** : compte d'un laboratoire (professionnel de type LABORATOIRE) ; saisit les résultats des analyses qui lui sont affectées.
- **Administrateur** : gère les comptes et le catalogue des médicaments.

1.5 Fonctionnalités principales

- Inscription, authentification (Keycloak) et gestion des profils.
- Dossier médical : groupe sanguin, allergies, antécédents, consultations.
- Gestion des autorisations d'accès au dossier par le patient.
- Rédaction d'ordonnances avec alertes d'allergies et d'interactions.
- Catalogue des médicaments, interactions et retrait d'un médicament du marché.
- Délivrance des ordonnances en pharmacie, une seule fois.
- Demande d'analyses et saisie des résultats par le laboratoire.
- Créneaux, prise et annulation de rendez-vous.

1.6 Périmètre

La plateforme concerne la **médecine générale** : les médecins sont des généralistes, sans gestion de spécialités. Les analyses sont des **analyses biologiques courantes** (sang, urine) ; l'imagerie médicale (radiographie, scanner…) est hors périmètre. La facturation, le remboursement CNAM et la gestion interne des établissements sont également hors périmètre. Les données sont **fictives**.

1 Présentation du projet — fonctionnement détaillé (1/2)

Pour chaque fonctionnalité : le problème qu'elle résout (pourquoi), son fonctionnement (comment) et les microservices concernés.

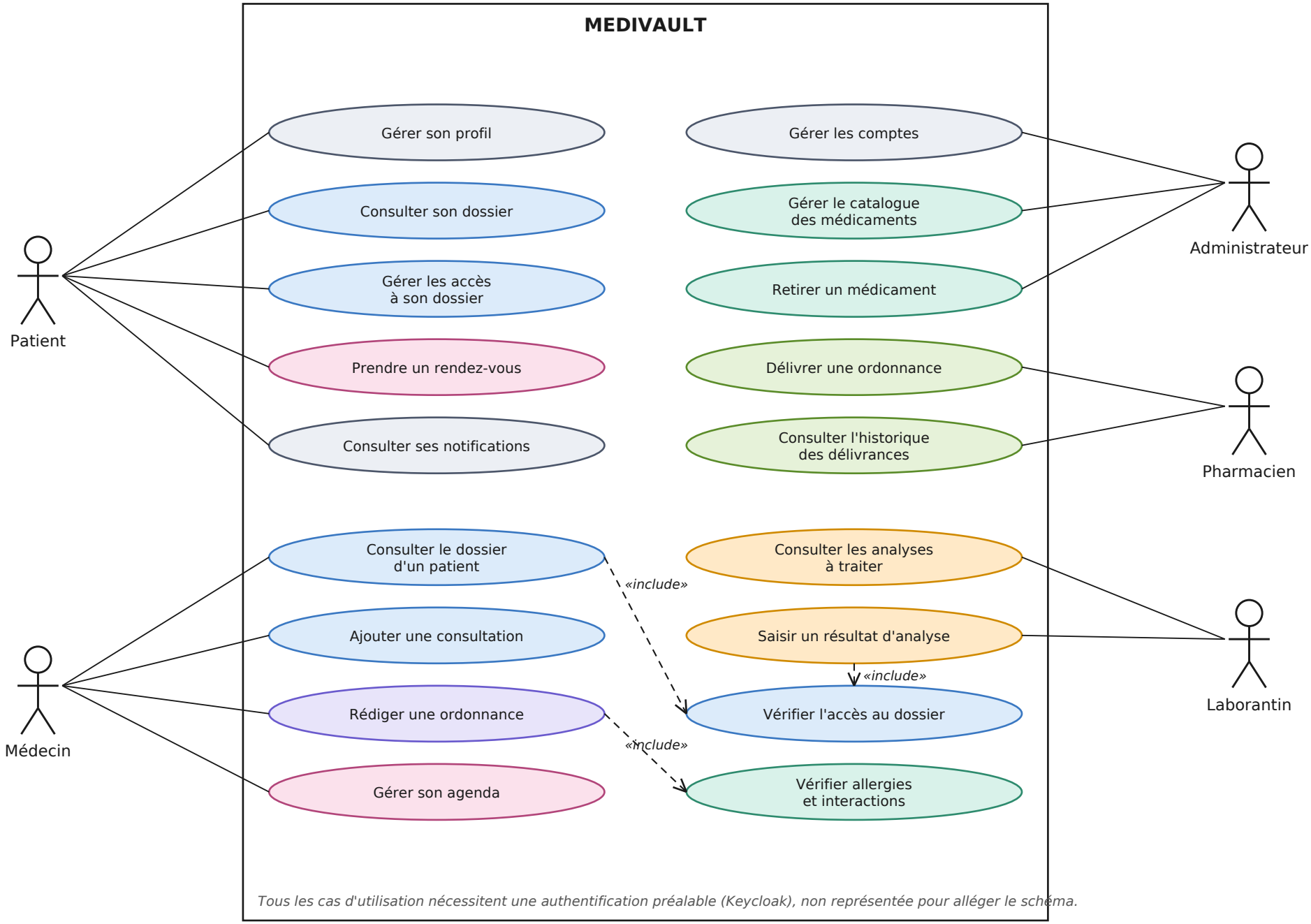
Fonctionnalité	Pourquoi ?	Comment ça marche ?	Services
1. Inscription et profils	Chaque acteur a des droits différents ; le patient doit savoir **où** et **quand** un médecin consulte.	L'utilisateur s'inscrit ; le user-service crée son compte et son rôle dans Keycloak. Un **professionnel** (médecin ou laboratoire) renseigne en plus son adresse, sa ville, ses **jours** et ses **heures de travail**.	user-service, Keycloak
2. Dossier médical unique	Les informations du patient sont dispersées entre plusieurs médecins et sur papier.	À l'inscription d'un patient, l'événement **UtilisateurCree** crée automatiquement son dossier (vide). Le dossier contient le groupe sanguin, les allergies, les antécédents et l'historique des consultations.	user, dossier
3. Contrôle des accès	Aujourd'hui, le patient ne décide pas qui consulte ses données médicales.	Le patient **accorde** ou **retire** l'accès à un professionnel (**médecin ou laboratoire**). À chaque ouverture du dossier ou saisie d'un résultat, le service vérifie que l'accès est **ACTIF**. Les dates d'accord et de retrait sont conservées (traçabilité).	dossier, analyse
4. Rendez-vous selon les disponibilités	Le patient ne sait pas quels jours ni à quelles heures le médecin ou le laboratoire reçoit.	Le rendezvous-service demande au user-service les jours et heures de travail du professionnel (**médecin** ou **laboratoire**) et **génère les créneaux**. Un créneau est libre tant qu'aucun rendez-vous confirmé ne lui est associé ; le patient en réserve un. Un rendez-vous au laboratoire fait passer l'analyse prescrite à EN_COURS et lui affecte ce laboratoire (**laboratoire_id**).	rendezvous, user, analyse
5. Consultation	Garder une trace fiable de chaque visite, accessible aux médecins autorisés.	Le médecin autorisé enregistre le motif, le diagnostic et ses observations. L'événement **ConsultationAjoutee** clôt le rendez-vous et crée les analyses éventuellement demandées.	dossier, rendezvous, analyse

Fonctionnalité	Pourquoi ?	Comment ça marche ?	Services
6. Ordonnance sécurisée	Un médecin peut prescrire, sans le savoir, un médicament dangereux pour le patient (allergie, interaction).	Avant validation, l'ordonnance récupère les **allergies** du patient (dossier) et demande au medicament-service de vérifier chaque médicament : s'il appartient à la **même classe thérapeutique** qu'une allergie, ou s'il présente une interaction **CONTRE_INDIQUEE**, la prescription est refusée (**409**) avec une alerte et des alternatives. Sinon, l'ordonnance est **VALIDEE** et reçoit un code unique.	ordonnance, dossier, medicament
7. Délivrance unique en pharmacie	Une ordonnance papier peut être réutilisée ou falsifiée.	Le pharmacien saisit le code ; le pharmacie-service vérifie auprès de l'ordonnance-service qu'elle est délivrable. La délivrance est **totale** ou **partielle**, et chaque ligne n'est délivrée qu'une seule fois. L'événement **OrdonnanceDelivree** met à jour l'ordonnance, l'historique du dossier et les statistiques (**nb_delivrances** du médicament).	pharmacie, ordonnance
8. Analyses	Les résultats papier arrivent tard et se perdent.	L'analyse prescrite pendant la consultation est créée **EN_ATTENTE**. Quand le patient réserve au laboratoire, elle passe **EN_COURS** et reçoit le **laboratoire_id**. Seul ce laboratoire, avec un accès **ACTIF** au dossier, saisit les résultats ; l'interprétation (NORMAL, BAS, ELEVE) est calculée selon la valeur de référence. **ResultatDisponible** ajoute le résultat au dossier et prévient le patient.	analyse, dossier, rendezvous, user
9. Retrait d'un médicament	Un médicament peut être retiré du marché (lot dangereux).	L'administrateur retire le médicament du catalogue : il ne peut plus être prescrit. L'événement **MedicamentRetire** est enregistré par le pharmacie-service (**MedicamentBloque**) : la délivrance est bloquée immédiatement, sans rappeler le medicament-service.	medicament, pharmacie
10. Notifications en temps réel	Le patient doit être informé sans avoir à vérifier lui-même.	Le user-service consomme via RabbitMQ les événements des autres services (accès, ordonnances, résultats, rendez-vous), enregistre la notification dans MongoDB et l'envoie immédiatement au navigateur (SSE).	user

1 Présentation du projet — scénario complet

Exemple suivi de bout en bout, utilisé pour la démonstration. Les noms et données sont fictifs.

N°	Acteur	Action	Ce que fait le système
1	Ahmed (patient)	S'inscrit sur MediVault.	user-service crée son compte dans Keycloak ; UtilisateurCree → dossier-service crée son dossier vide.
2	Dr Karim (médecin)	S'inscrit : cabinet à Ariana, du lundi au vendredi, 9 h – 17 h.	Profil professionnel enregistré (type MEDECIN, adresse du cabinet, jours et heures de travail).
3	Ahmed	Accorde l'accès à son dossier au Dr Karim.	AccesDossier ACTIF (date d'accord) ; AccesAccorde → notification.
4	Ahmed	Réserve un rendez-vous mardi à 10 h.	rendezvous-service lit les disponibilités du médecin, réserve le créneau ; RendezVousConfirme → notification.
5	Dr Karim	Ouvre le dossier d'Ahmed.	Accès vérifié (ACTIF) ; il voit l'allergie à la pénicilline.
6	Dr Karim	Ajoute la consultation (angine) et demande un bilan sanguin.	ConsultationAjoutee → rendez-vous TERMINE ; analyse créée EN_ATTENTE.
7	Dr Karim	Prescrit de l'Augmentin.	Augmentin appartient à la classe des pénicillines → ordonnance refusée (409), alerte et alternative proposée.
8	Dr Karim	Prescrit l'alternative proposée.	Ordonnance VALIDEE, code ORD-2026-0123 ; OrdonnanceCreee → notification.
9	Pharmacien	Saisit le code et délivre l'ordonnance.	Vérification puis délivrance TOTALE ; OrdonnanceDelivree → ordonnance DELIVREE, historique du dossier, statistiques.
9b	Ahmed	Accorde l'accès à son dossier au laboratoire BioLab (Ariana) et y réserve un rendez-vous.	AccesDossier ACTIF pour BioLab ; créneau généré selon les horaires du laboratoire ; RendezVousConfirme → analyse EN_COURS (laboratoire_id = BioLab), notification.
10	Laborantin (BioLab)	Saisit la glycémie : 1,30 g/L.	Laboratoire affecté et accès ACTIF vérifiés ; interprétation ELEVE ; ResultatDisponible → résultat ajouté au dossier, notification en temps réel.
11	Ahmed	Retire l'accès au Dr Karim.	AccesDossier RETIRE (date de retrait) : le médecin ne peut plus ouvrir le dossier.


MEDIVAULT
Gérer son profil
Consulter son dossier
Gérer les accès à son dossier
Prendre un rendez-vous
Consulter ses notifications
Gérer les comptes
Gérer le catalogue des médicaments
Retirer un médicament
Délivrer une ordonnance
Consulter l'historique des délivrances
Consulter le dossier d'un patient
Ajouter une consultation
Rédiger une ordonnance
Gérer son agenda
Consulter les analyses à traiter
Saisir un résultat d'analyse
Vérifier l'accès au dossier
Vérifier allergies et interactions
«include»
«include»
«include»
Patient
Médecin
Administrateur
Pharmacien
Laborantin
Tous les cas d'utilisation nécessitent une authentification préalable (Keycloak), non représentée pour alléger le schéma.

3 Découpage en microservices

Le découpage suit les domaines métier (contextes bornés) et non les tables. Chaque gestion comprend une partie back-end (microservice) et une partie front-end (pages), réalisées par le même membre. Chaque microservice possède sa propre base ; aucune base n'est partagée.

Gestion / microservice	Responsabilité	Données gérées	Base	Pages front-end
Utilisateurs (commun) **user-service**	Microservice avancé : comptes créés dans Keycloak (Admin API), profil des professionnels — médecins et laboratoires (adresse, jours et heures de travail), centre de notifications en temps réel (SSE).	Utilisateur, Professionnel, Notification	MongoDB	Inscription, connexion, profil, notifications, administration des comptes
Dossier médical **dossier-service**	Dossier du patient (groupe sanguin, allergies, antécédents), consultations et autorisations d'accès (médecin ou laboratoire).	Dossier, Consultation, AccesDossier	MySQL	Dossier patient, ajout de consultation, gestion des accès
Ordonnances **ordonnance-service**	Rédaction et cycle de vie des ordonnances, contrôle avant validation.	Ordonnance, LigneOrdonnance	MySQL	Rédaction avec alertes, liste et détail des ordonnances
Médicaments **medicament-service**	Catalogue (DCI, famille thérapeutique), interactions, vérification de compatibilité, retrait, statistiques.	Medicament, Interaction	H2	Catalogue, interactions, retrait d'un médicament
Pharmacie **pharmacie-service**	Recherche et délivrance unique (totale ou partielle) des ordonnances, blocage des médicaments retirés.	Pharmacie, Delivrance, MedicamentBloque	H2	Recherche par code, délivrance, historique
Analyses **analyse-service**	Analyses demandées, affectation au laboratoire du rendez-vous, saisie des résultats.	Analyse, ResultatAnalyse	H2	Analyses à traiter, saisie des résultats, résultats du patient
Rendez-vous **rendezvous-service**	Créneaux générés selon les horaires du médecin ou du laboratoire, prise et annulation de rendez-vous.	Creneau, RendezVous	H2	Agenda, prise de rendez-vous, mes rendez-vous

Justification des choix de bases de données

Base	Utilisée par	Justification
H2	Médicaments, Pharmacie, Analyses, Rendez-vous	Base relationnelle embarquée et légère, adaptée à des données structurées simples. Le catalogue des médicaments est rechargé au démarrage par data.sql.
MySQL	Dossier médical, Ordonnances	Données médicales et légales qui doivent être conservées durablement (persistance hors de l'application), transactions et relations (consultations, lignes d'ordonnance).
MongoDB	Utilisateurs (commun)	Utilisateurs aux champs différents selon le rôle (un professionnel a en plus son adresse et ses horaires : héritage) et notifications nombreuses aux champs optionnels : un modèle documentaire est plus adapté. Choix MongoDB du microservice commun.

Technologies

Couche	Technologie	Rôle
Microservices	Spring Boot 3.5, Spring Data JPA, Spring Data MongoDB	Logique métier et persistance de chaque service
Infrastructure	Spring Cloud Gateway, Eureka, Config Server	Point d'entrée unique, découverte, configuration centralisée
Communication	OpenFeign, RabbitMQ (Spring AMQP), Resilience4j, Server-Sent Events (SSE)	Appels REST, événements métier asynchrones (files durables), circuit breaker, notifications temps réel
Sécurité	Keycloak (OAuth2 / JWT)	Authentification et rôles : PATIENT, MEDECIN, PHARMACIEN, LABORANTIN, ADMIN
Front-end · Déploiement	Angular · Docker, Docker Compose	Interface des acteurs · lancement global de la solution
Documentation · Gestion	Swagger / OpenAPI · GitHub, Trello	Contrats d'API · suivi des tâches et du code

3 Le microservice commun : user-service (technologie avancée)

Rôle

Le **user-service** est développé par toute l'équipe car **tous les autres services en dépendent** : chaque action concerne un utilisateur (patient, médecin, pharmacien, laborantin, administrateur), et chaque événement important doit être notifié à la bonne personne.

Il joue le rôle de **l'accueil de la plateforme** : il inscrit les utilisateurs, leur attribue leurs droits, et les prévient en direct de tout ce qui les concerne.

Pourquoi est-il « avancé » ?

Les six autres microservices reposent sur le même modèle (Spring Data JPA, base relationnelle, CRUD). Le user-service introduit **des technologies et des mécanismes que les autres n'utilisent pas** :

- **Base NoSQL MongoDB** : tous les utilisateurs sont dans la même collection ; un **professionnel** (médecin ou laboratoire, classe fille de Utilisateur) possède en plus son adresse, sa ville, ses jours et heures de travail. Les autres utilisateurs n'ont pas ces champs : aucune colonne vide.
- **Intégration Keycloak (Admin API)** : à l'inscription, le compte et le rôle sont créés automatiquement dans Keycloak ; la désactivation d'un compte est répercutée.
- **Centre de notifications orienté événements** : le service qui consomme le plus d'événements (accès, ordonnances, résultats, rendez-vous), chacun reçu dans sa propre file RabbitMQ.
- **Temps réel (Server-Sent Events)** : les notifications sont poussées immédiatement vers le front-end, sans rechargement de page.
- **Champs optionnels** : une notification ne contient que l'identifiant qui la concerne (ordonnance_id, analyse_id ou rendezvous_id) ; les champs absents ne sont pas stockés.
- **Fonctions MongoDB** : index TTL (suppression automatique des notifications de plus de 90 jours) et index texte (recherche d'utilisateurs).

Exemple de documents MongoDB

```
{ "nom": "Karim", "role": "MEDECIN", "type": "MEDECIN",
  "ville": "Ariana",
  "adresse": "12 av. de la Liberté",
  "jours_travail": ["MONDAY", …, "FRIDAY"],
  "heure_debut": "09:00", "heure_fin": "17:00" }

{ "nom": "Ahmed", "role": "PATIENT",
  "email": "ahmed@mail.com" }
```

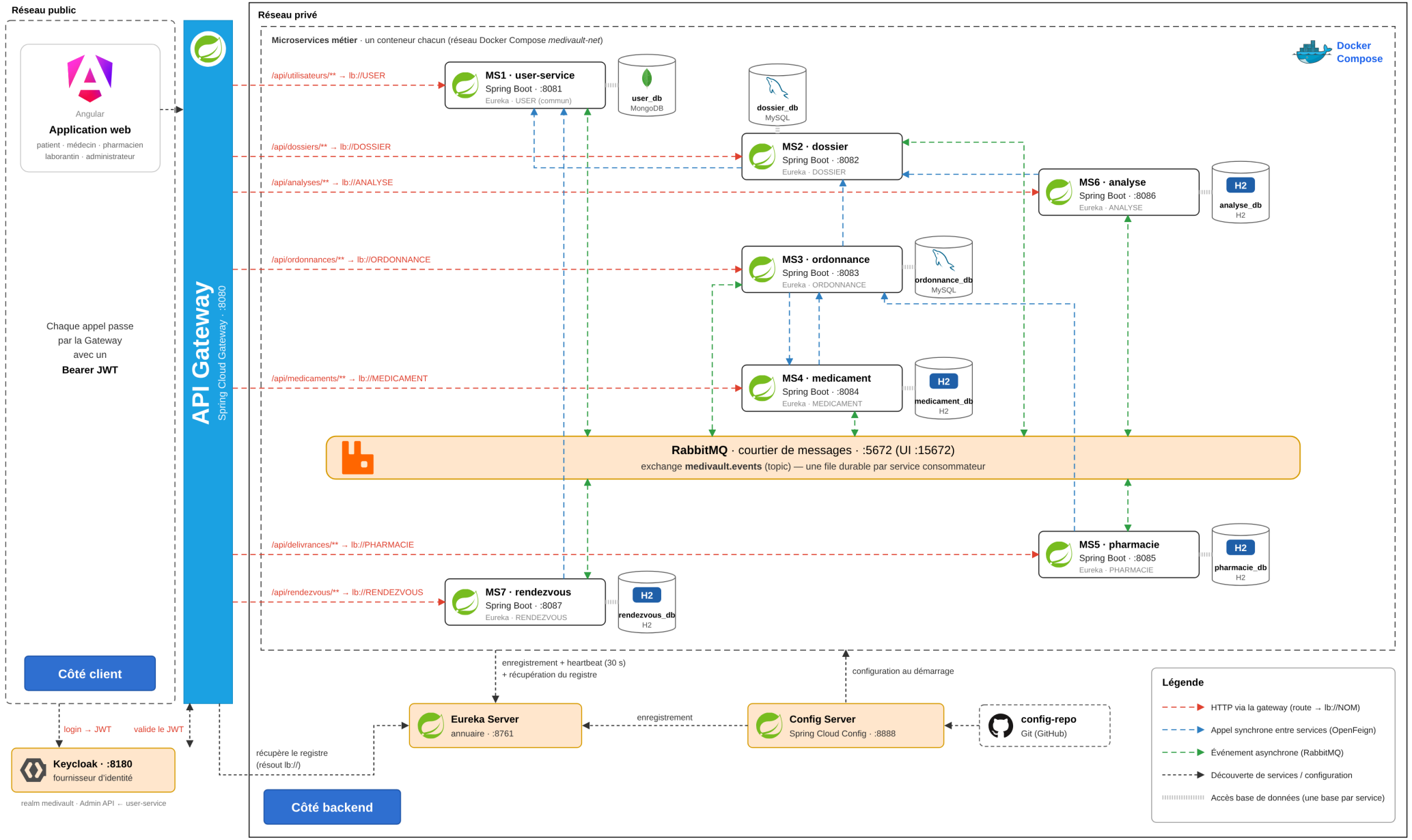
Fonctionnement d'une notification

- Le laborantin saisit un résultat : `ResultatDisponible` est publié sur RabbitMQ (exchange `medivault.events`).
- Le user-service consomme l'événement et identifie le patient concerné.
- La notification est enregistrée dans MongoDB.
- Elle est envoyée immédiatement au navigateur du patient via SSE.
- Le patient voit « Un nouveau résultat d'analyse est disponible » sans aucune action.

Synthèse

Les autres services gèrent **leurs propres données métier** ; le user-service gère **les personnes, leurs droits et leur information en temps réel**, à l'aide de MongoDB, de Keycloak, des événements des autres services et des Server-Sent Events.

Vue d'ensemble de l'architecture MediVault



user-service (commun)

MongoDB

«document» Utilisateur

- id : String
- keycloak_id : String
- nom : String
- prenom : String
- email : String
- telephone : String
- role : Role
- actif : boolean

+ desactiver() : void

1 — reçoit — 0..*

«document» Notification

- id : String
- message : String
- type : TypeNotification
- ordonnance_id : Long
- analyse_id : Long
- rendezvous_id : Long
- lue : boolean
- date_creation : Instant

+ marquerLue() : void

hérite de

«document» Professionnel

- type : TypeProfessionnel
- numero_agrement : String
- adresse : String
- ville : String
- jours_travail : List<DayOfWeek>
- heure_debut : LocalTime
- heure_fin : LocalTime

+ travailleLe(jour) : boolean

dossier-service

MySQL

«entity» Dossier

- id : Long
- patient_id : Long
- groupe_sanguin : GroupeSanguin
- allergies : List<String>
- antecedents : String
- date_creation : LocalDateTime

+ ajouterAllergie(a) : void
+ estAutorise(professionnelId) : boolean

1 — *contient* — 0..*

«entity» Consultation

- id : Long
- medecin_id : Long
- date_consultation : LocalDateTime
- motif : String
- diagnostic : String
- observations : String

+ modifier(medecinId) : void

1 — *autorise* — 0..*

«entity» AccesDossier

- id : Long
- professionnel_id : Long
- statut : StatutAcces
- date_accord : LocalDateTime
- date_retrait : LocalDateTime

+ retirer() : void

ordonnance-service

MySQL

«entity» Ordonnance

- id : Long
- code : String {unique}
- patient_id : Long
- medecin_id : Long
- consultation_id : Long
- date_emission : LocalDate
- date_expiration : LocalDate
- statut : StatutOrdonnance

+ estDelivrable() : boolean

contient 1 — 1..*

«entity» LigneOrdonnance

- id : Long
- medicament_id : Long
- nom_medicament : String
- posologie : String
- duree_jours : int
- quantite : int
- quantite_delivree : int

+ resteADelivrer() : int

medicament-service

H2

«entity» Medicament

- id : Long
- nom_commercial : String
- dci : String
- classe_therapeutique : String
- forme : FormeMedicament
- dosage : String
- nb_delivrances : int

+ estDeLaClasse(c) : boolean

medicament_a 1 — 0..* ; 1 — 0..* *medicament_b*

«entity» Interaction

- id : Long
- gravite : Gravite
- description : String

+ estBloquante() : boolean

pharmacie-service

H2

«entity» Pharmacie

- id : Long
- nom : String
- adresse : String
- gouvernorat : String
- code_agrement : String

+ delivrer(o) : Delivrance

effectue 1 — 0..*

«entity» Delivrance

- id : Long
- ordonnance_id : Long
- pharmacien_id : Long
- date_delivrance : LocalDateTime
- type : TypeDelivrance
- lignes_delivrees : List<Long>

+ estComplete() : boolean

alimentée par l'événement MedicamentRetire

«entity» MedicamentBloque

- id : Long
- medicament_id : Long
- motif : String
- date_retrait : LocalDateTime

+ concerne(medicamentId) : boolean

analyse-service

H2

«entity» Analyse

- id : Long
- patient_id : Long
- medecin_id : Long
- type_analyse : String
- statut : StatutAnalyse
- laboratoire_id : Long

+ terminer() : void

produit 1 — 1..*

«entity» ResultatAnalyse

- id : Long
- parametre : String
- valeur : double
- unite : String
- valeur_reference : String
- interpretation : Interpretation

+ interpreter() : void

rendezvous-service

H2

«entity» Creneau

- id : Long
- professionnel_id : Long
- type_professionnel : TypeProfessionnel
- date_debut : LocalDateTime
- date_fin : LocalDateTime

+ estLibre() : boolean

réservé par 1 — 0..1

«entity» RendezVous

- id : Long
- patient_id : Long
- motif : String
- statut : StatutRdv

+ annuler() : void
+ terminer() : void

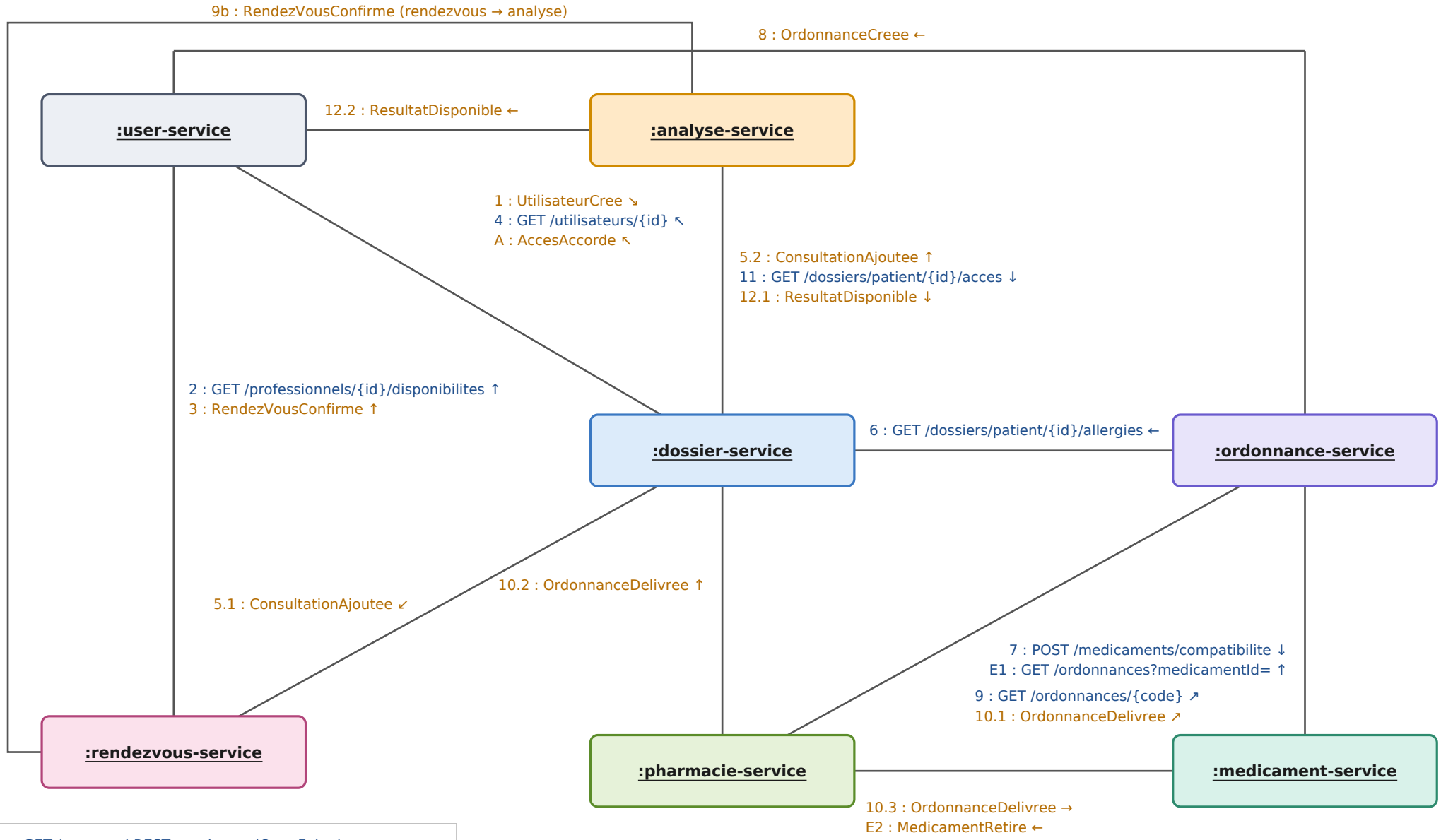
Énumérations

Énumération	Service	Valeurs
Role	user	`PATIENT, MEDECIN, PHARMACIEN, LABORANTIN, ADMIN`
TypeNotification	user	`RENDEZ_VOUS, ORDONNANCE, RESULTAT, ACCES`
GroupeSanguin	dossier	`A_POS, A_NEG, B_POS, B_NEG, AB_POS, AB_NEG, O_POS, O_NEG`
StatutAcces	dossier	`ACTIF, RETIRE`
StatutOrdonnance	ordonnance	`EN_ATTENTE_VERIFICATION, VALIDEE, PARTIELLEMENT_DELIVREE, DELIVREE, EXPIREE, ANNULEE`
FormeMedicament	medicament	`COMPRIME, GELULE, SIROP, INJECTABLE, POMMADE`
Gravite	medicament	`FAIBLE, MODEREE, GRAVE, CONTRE_INDIQUEE`
TypeDelivrance	pharmacie	`TOTALE, PARTIELLE`
StatutAnalyse · Interpretation	analyse	`EN_ATTENTE, EN_COURS, TERMINEE · NORMAL, BAS, ELEVE`
TypeProfessionnel · StatutRdv	user, rendezvous	`MEDECIN, LABORATOIRE · CONFIRME, ANNULE, TERMINE`

Règles de gestion

N°	Règle	Service
RG1	Un patient possède un et un seul dossier, créé automatiquement à son inscription.	dossier
RG2	Un professionnel (médecin ou laboratoire) ne lit ou n'enrichit un dossier que s'il possède un accès ACTIF accordé par le patient.	dossier
RG3	Une consultation n'est jamais supprimée ; seul le médecin qui l'a rédigée peut la modifier.	dossier
RG4	Une ordonnance est refusée si un médicament appartient à la même classe thérapeutique qu'une allergie du patient (les allergies sont choisies parmi les classes thérapeutiques du catalogue), ou présente une interaction CONTRE_INDIQUEE.	ordonnance / medicament
RG5	Une ordonnance n'est délivrable que si elle est VALIDEE ou PARTIELLEMENT_DELIVREE et non expirée ; une ligne n'est jamais délivrée deux fois.	pharmacie / ordonnance
RG6	Un médicament retiré du catalogue ne peut plus être prescrit ; le pharmacie-service l'enregistre dans MedicamentBloque (MedicamentRetire) et refuse sa délivrance.	medicament / pharmacie
RG7	Un résultat d'analyse n'est saisi que pour une analyse EN_COURS (laboratoire affecté) ; l'interprétation est calculée selon la valeur de référence.	analyse
RG8	Les créneaux sont générés dans les jours et heures de travail du professionnel (médecin ou laboratoire) ; un créneau est libre tant qu'aucun rendez-vous CONFIRME ne lui est associé.	rendezvous
RG9	Un professionnel (médecin ou laboratoire) possède les attributs d'un utilisateur, plus son adresse et ses horaires (héritage).	user
RG10	Seul le laboratoire affecté à l'analyse (laboratoire_id, issu du rendez-vous confirmé) saisit son résultat, et seulement s'il possède un accès ACTIF au dossier.	analyse / dossier

Lecture du diagramme : ◆ composition (la partie n'existe pas sans le tout) ; △ héritage (un Professionnel est un Utilisateur) ; xxx_id = identifiant d'un autre service, sans clé étrangère entre bases.



n : GET /… appel REST synchrone (OpenFeign)
n : Événement message asynchrone (RabbitMQ)
Numéros = ordre du scénario ; n.1, n.2 = en parallèle
A, E1, E2 = accès accordé / retrait d'un médicament

6 Interactions — détail par microservice

Synthèse : chaque microservice réalise au moins un appel synchrone, publie et consomme au moins un événement

Microservice	Appel synchrone (OpenFeign)	Publie	Consomme
Utilisateurs (commun)	→ Keycloak (Admin API) : création du compte et attribution du rôle	UtilisateurCree	AccesAccorde, OrdonnanceCreee, ResultatDisponible, RendezVousConfirme → notifications
Dossier médical	→ Utilisateurs : le professionnel existe-t-il et est-il actif ?	ConsultationAjoutee, AccesAccorde	UtilisateurCree (dossier vide), ResultatDisponible, OrdonnanceDelivree (historique)
Ordonnances	→ Dossier : allergies du patient ; → Médicaments : compatibilité	OrdonnanceCreee	OrdonnanceDelivree (statut DELIVREE)
Médicaments	→ Ordonnances : ordonnances actives contenant le médicament	MedicamentRetire	OrdonnanceDelivree (nb_delivrances : statistiques de consommation)
Pharmacie	→ Ordonnances : ordonnance valide et non délivrée ?	OrdonnanceDelivree	MedicamentRetire (MedicamentBloque : blocage de la délivrance)
Analyses	→ Dossier : le laboratoire a-t-il un accès ACTIF au dossier ?	ResultatDisponible	ConsultationAjoutee (analyse EN_ATTENTE), RendezVousConfirme (rdv labo → EN_COURS + laboratoire_id)
Rendez-vous	→ Utilisateurs : jours et heures de travail du médecin ou du laboratoire	RendezVousConfirme	ConsultationAjoutee (rendez-vous terminé)

Comportement en cas de panne du service appelé

Appel synchrone	Si le service appelé est indisponible	Justification
Dossier → Utilisateurs	Accès refusé (503)	Fail-closed : aucune donnée médicale sans vérification
Ordonnances → Dossier / Médicaments	Ordonnance enregistrée « EN_ATTENTE_VERIFICATION », non délivrable	La sécurité du patient prime sur la rapidité
Médicaments → Ordonnances	Retrait publié quand même ; liste des ordonnances recalculée plus tard	Le retrait d'un médicament dangereux ne doit pas être bloqué
Pharmacie → Ordonnances	Délivrance refusée (503), nouvel essai proposé	Évite une double délivrance
Analyses → Dossier	Saisie refusée (503)	Fail-closed sur les données médicales
Rendez-vous → Utilisateurs	Créneaux non générés, message « réessayer plus tard »	Évite des créneaux hors des horaires du professionnel

Résilience : circuit breaker et fallback avec Resilience4j. Côté asynchrone, messages RabbitMQ persistants et une file durable par service consommateur : un service arrêté retrouve ses messages dans sa file et les traite à son redémarrage (acquittement après traitement).

Service	Méthode	Endpoint	Description	Codes HTTP
user-service	POST	/api/utilisateurs	Créer un compte	201, 400, 409
	GET	/api/utilisateurs/{id}	Consulter un utilisateur (appelé par dossier)	200, 404
	GET	/api/professionnels?type=&ville=	Rechercher un médecin / laboratoire	200
	GET	/api/professionnels/{id}/disponibilites	Jours et heures de travail (appelé par rendezvous)	200, 404
	GET	/api/utilisateurs/{id}/notifications	Notifications	200
	GET	/api/notifications/stream	Flux temps réel (SSE)	200
	PATCH	/api/notifications/{id}	Marquer comme lue	200, 404
dossier-service	GET	/api/dossiers/patient/{patientId}	Consulter un dossier	200, 403, 404, 503
	POST	/api/dossiers/{id}/consultations	Ajouter une consultation	201, 400, 403
	GET	/api/dossiers/patient/{patientId}/allergies	Allergies du patient (appelé par ordonnance)	200, 404
	GET	/api/dossiers/patient/{patientId}/acces?professionnelId=	Accès ACTIF du professionnel ? (appelé par analyse)	200, 404
	POST	/api/dossiers/{id}/acces	Accorder un accès (médecin ou laboratoire)	201, 400, 404, 409
	PATCH	/api/dossiers/{id}/acces/{professionnelId}	Retirer un accès (statut RETIRE, date conservée)	200, 404
ordonnance-service	POST	/api/ordonnances	Rédiger (avec vérification)	201, 400, 409
	GET	/api/ordonnances/{code}	Consulter par code (appelé par pharmacie)	200, 404
	GET	/api/ordonnances?patientId=	Ordonnances d'un patient	200
	GET	/api/ordonnances?medicamentId=&statut=	Recherche par médicament (appelé par medicament)	200
medicament-service	GET	/api/medicaments	Catalogue	200
	POST	/api/medicaments	Ajouter un médicament	201, 400
	POST	/api/medicaments/compatibilite	Compatibilité allergies / interactions (appelé par ordonnance)	200, 400
	DELETE	/api/medicaments/{id}	Retirer du catalogue	204, 404

Tous les endpoints sont exposés via l'API Gateway, protégés par JWT (Keycloak) et documentés avec Swagger / OpenAPI.
POST réussi → 201 + en-tête Location ; erreurs au format Problem Details (RFC 9457) ; 409 = règle métier violée (ex. ordonnance refusée pour allergie).

Service	Méthode	Endpoint	Description	Codes HTTP
pharmacie-service	GET	/api/pharmacies	Lister les pharmacies	200
	POST	/api/delivrances	Délivrer une ordonnance	201, 404, 409, 503
	GET	/api/delivrances?pharmacieId=	Historique	200
	GET	/api/delivrances/ordonnance/{id}	Délivrance d'une ordonnance	200, 404
analyse-service	GET	/api/analyses?statut=EN_COURS&laboratoireId=	Analyses à traiter par le laboratoire	200
	POST	/api/analyses/{id}/resultats	Saisir un résultat (laboratoire affecté, accès ACTIF)	201, 400, 403, 503
	GET	/api/analyses/patient/{patientId}	Analyses d'un patient	200
	GET	/api/analyses/{id}	Détail	200, 404
rendezvous-service	POST	/api/creneaux	Générer les créneaux d'un professionnel	201, 400, 404, 503
	GET	/api/creneaux?professionnelId=&date=	Créneaux libres	200
	POST	/api/rendezvous	Prendre un rendez-vous	201, 409
	PATCH	/api/rendezvous/{id}	Annuler (statut ANNULE)	200, 404, 409

Événements métier (RabbitMQ : exchange topic medivault.events, une file durable par service consommateur)

Événement	Clé de routage	Producteur	Consommateurs	Payload
UtilisateurCree	utilisateur.cree	user	dossier	utilisateurId, role, nom
AccesAccorde	acces.accorde	dossier	user	dossierId, patientId, professionnelId
ConsultationAjoutee	consultation.ajoutee	dossier	rendezvous, analyse	consultationId, patientId, medecinId, analysesDemandees
OrdonnanceCreee	ordonnance.creee	ordonnance	user	ordonnanceId, code, patientId
OrdonnanceDelivree	ordonnance.delivree	pharmacie	ordonnance, dossier, medicament	ordonnanceId, patientId, lignesDelivrees, type
MedicamentRetire	medicament.retire	medicament	pharmacie	medicamentId, motif, date
ResultatDisponible	resultat.disponible	analyse	dossier, user	analyseId, patientId, laboratoireId, typeAnalyse
RendezVousConfirme	rendezvous.confirme	rendezvous	user, analyse	rendezVousId, patientId, professionnelId, typeProfessionnel, date

8 Organisation du travail

8.1 Répartition des tâches (chaque membre réalise le back-end et le front-end de sa gestion)

Membre	Gestion	Back-end	Front-end
Eya	Dossier médical	dossier-service (MySQL), accès au dossier	Dossier, consultations, accès
Amine	Ordonnances	ordonnance-service (MySQL), vérification avant validation	Rédaction avec alertes, liste
Fedia	Médicaments	medicament-service, interactions, retrait	Catalogue, interactions
Aziz	Pharmacie	pharmacie-service, délivrance unique, médicaments bloqués	Recherche, délivrance, historique
Taha	Analyses	analyse-service, affectation au laboratoire, résultats	Analyses à traiter, saisie, mes résultats
Yazid	Rendez-vous	rendezvous-service, créneaux	Agenda, prise de rendez-vous
Toute l'équipe	Utilisateurs (commun)	user-service (MongoDB), Keycloak, notifications	Inscription, connexion, profil, notifications

8.2 Parties communes

- **Yazid** : présentation et suivi des tâches.
- **Amine** : RabbitMQ (exchange, clés de routage, format des événements).
- **Aziz** : API Gateway, Eureka, Config Server ; modèle MongoDB du user-service.
- **Fedia** : dépôt Git, README et diagrammes.
- **Eya** : Keycloak (realm, rôles) et intégration Admin API du user-service.
- **Taha** : socle Angular, Docker Compose et notifications temps réel (SSE).

8.3 Outil de gestion de projet

Tableau **Trello** : une carte par tâche, un responsable, une échéance, colonnes « À faire / En cours / Terminé ». Chaque membre effectue des commits réguliers dans son propre microservice.

8.4 Organisation du dépôt Git

```
medivault-4dasi4-keystone/
├── Backend/
│   ├── eureka-server/ · config-server/ · api-gateway/
│   ├── user-service/ · dossier-service/ · ordonnance-service/
│   ├── medicament-service/ · pharmacie-service/
│   └── analyse-service/ · rendezvous-service/
├── Frontend/
├── Documentation/
│   ├── Presentation/
│   ├── Diagrams/
│   └── MediVault_Dossier_Technique.pdf
└── README.md
```

8.5 Planning prévisionnel

Séance	Objectif	Livrable
S1 – S3	Cadrage, choix du sujet, découpage, contrats d'API	Conception
S4	Validation 1 : conception, dépôt Git, projets créés qui démarrent	Présentation + dépôt
S5	API Gateway, CRUD de chaque microservice, Swagger	Endpoints documentés
S6	Communication synchrone (OpenFeign, Resilience4j)	Appels inter-services
S7	Communication asynchrone (RabbitMQ)	Événements publiés / consommés
S8	Sécurité Keycloak (rôles, JWT)	Accès sécurisés
S9	Dockerisation, intégration du front-end	docker-compose fonctionnel
S10	Démonstration et soutenance	Application complète

8.6 Principaux risques

Risque	Mesure prévue
Dépendances entre services (un membre bloque les autres)	Contrats d'API figés dès la S4, données simulées en attendant
Complexité de Keycloak et de la sécurité	Un référent sécurité, rôles simples, mise en place progressive
Intégration tardive des services	Lancement commun de tous les services chaque semaine
Base de médicaments trop large	Catalogue réduit (30 médicaments, 10 interactions) chargé par data.sql
Contributions inégales	Tâches visibles dans l'outil de gestion, commits de chaque membre